The Midnight Queue

Ranking suspicious transactions at scale

Mutasim ur Rehman | 24i-2514 | Section B
FAST-NUCES, Islamabad | Design and Analysis of Algorithms | Assignment 2

0. Use of AI tools and other sources

I used AI coding agents, including Codex, to help with writing and simplifying Python code, debugging, preparing plots, and improving the report. I also took help from online sources such as Stack Overflow for coding questions and from course material for algorithm theory. These tools and references supported the work; I remain responsible for the submitted code, results, and explanations.

Project overview

The task is to rank transactions when analysts can review only a limited number of cases. My seed is 2514, so the shift capacity is K = 25 + (2514 mod 226) = 53. I compare full sorting, top-K selection, and updates as new transactions arrive.

A. Dataset and ranking rule

The file contains 284,807 transactions, including 492 labelled frauds. I standardise V1 to V28 over the full file, add their squared z-scores to obtain risk, and assign band = min(9, max(0, floor($\log_2$(risk)))). The ranking is **band descending, Amount descending, Time ascending, then txn_id ascending**. Class is used only for evaluation.

Band	Transactions	Share	Frauds
0	0	0.00%	0
1	0	0.00%	0
2	21,156	7.43%	0
3	107,410	37.71%	13
4	113,116	39.72%	30
5	32,084	11.27%	23
6	6,006	2.11%	56
7	2,797	0.98%	97
8	1,376	0.48%	112
9	862	0.30%	161

There are 32,767 distinct amounts. The ten most common amounts account for 16.29% of records. Time never decreases between consecutive rows, so the raw file meets C1; this does not mean it is already in triage order.

B. Implementation and testing

I implemented insertion, merge, heap, and both quicksort variants, along with a size-K heap and a bounded sorted buffer. The hardened quicksort uses a median-of-three pivot, three-way partitioning, and recursion on the smaller side. These changes limit stack depth and help with duplicate bands, but its worst-case time is still $O(n^2)$.

The current unittest suite has 15 test methods with subtests for algorithms, conditions, and ranking rules. It covers empty and single inputs, ties, top-K output, stable sorting, heap updates, and recursion failure. Full-key outputs are compared with sorted(); for C6, tied items may appear in different orders in unstable sorts.

C. Full-ranking experiments

I used six conditions: arrival order (C1), shuffled (C2), pre-ranked (C3), reversed (C4), 1% displaced (C5), and band-only comparison (C6). The four sizes are 1,000, 10,000, 100,000, and 284,807. Each algorithm receives a fresh copy. Timing excludes loading and risk calculation.

The table below uses the saved results.csv. It predates the code simplification: many small runs use five trials, expensive 10,000-row runs use three, and full-file entries use one. Some DNF points were skipped and extrapolated rather than timed to the limit. This falls short of the specified five-run median and should be rerun for final submission.

Full file: seconds	C1	C2	C3	C4	C5	C6
Insertion Sort	DNF*	DNF*	0.218	DNF*	115.159	DNF*
Merge Sort	3.640	3.775	2.878	2.825	3.236	2.488
Naive QuickSort	2.889	2.761	DNF*	DNF*	31.637	DNF*
Hardened QuickSort	3.606	3.754	73.094	62.775	6.166	0.191
Heap Sort	4.708	4.534	3.527	3.510	3.703	2.331

** DNF values are labelled extrapolations in results.csv; they are not completed full-file timings.*

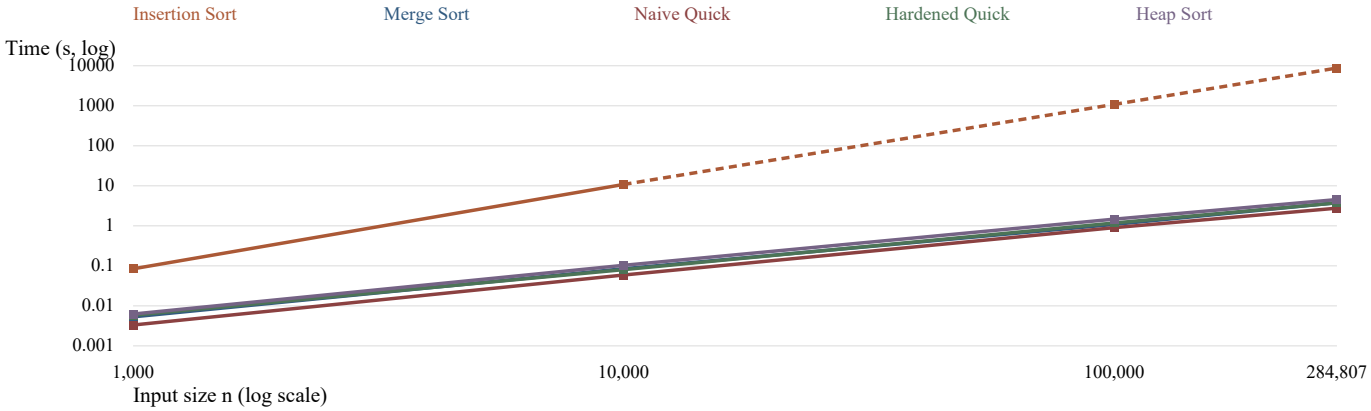


Figure 1. C2 runtime from results.csv. Both axes use log scales. Dashed sections are extrapolated DNF points; solid sections join completed runs.

C2, full file	Comparisons	Moves	Auxiliary space	Stable?
Insertion Sort	20,265,189,512*	20,273,302,637*	O(1)	Yes
Merge Sort	4,804,402	10,343,704	O(n)	Yes
Naive QuickSort	6,180,810	6,116,374	O(n) worst	No
Hardened QuickSort	8,778,624	11,612,318	O(log n)	No
Heap Sort	9,448,826	9,819,702	O(1)	No

What I learned

Insertion sort is very fast on C3 because it does not need to shift items. However, moving just 1% of the full file still creates many inversions, and C5 takes about 115 seconds. Nearly ranked data is not automatically a small job for insertion sort.

Naive quicksort is fastest on saved C1 and C2 full-file runs, but it performs badly on ordered input. Merge sort gives a more consistent choice across conditions. The hardened quicksort is best on C6, yet takes 73.09 seconds on C3; median-of-three is not a guarantee of balanced partitions.

Measurement notes

A comparison means one call to the ranking comparator. The current counters treat a non-self swap as two moves and also count saved insertion items, shifts, and output writes. Memory figures count list slots rather than total Python process memory. Original tournament hardware was not recorded, so these timings should not be treated as portable performance claims.

D. Selecting the shift queue

For a queue of 53 cases, sorting every transaction does more work than the output requires. I compare merge sort followed by slicing, a heap that keeps the best K records, and a bounded insertion buffer. The heap root is the weakest survivor, so most new records can be rejected after one comparison.

Method	K	Time (s)	Comparisons	Moves
Full merge sort	53	3.8601	4,793,437	10,343,704
Size-K min-heap	53	0.1094	289,614	5,053
Bounded buffer	53	0.0847	287,848	14,688
Full merge sort	2068	3.7957	4,793,437	10,343,704
Size-K min-heap	2068	0.2338	525,295	249,171
Bounded buffer	2068	1.4957	419,087	12,343,548

Table 3. Fresh single-run checks on the full file in arrival order, using the current code. Copies and loading are outside the timer. These are checks, not five-run benchmark medians.

Under the standard key, K = 53 captures 0 frauds. The smallest queue containing half of all frauds is K = 2,068, which captures 246 cases. High risk-band membership alone does not guarantee that a transaction is fraudulent, and the Amount tie-breaker affects which cases fit in a small queue.

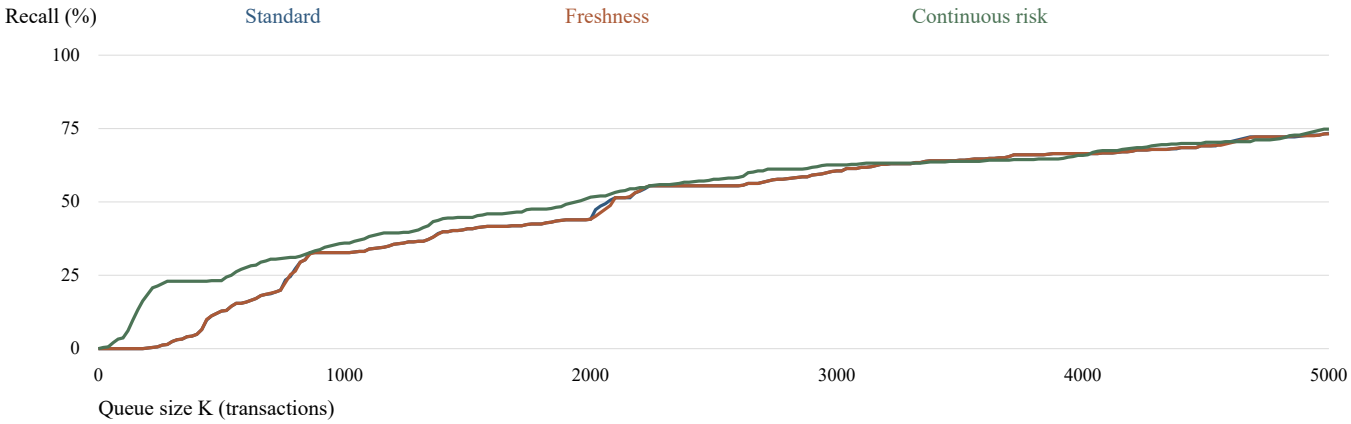


Figure 2. Fraud recall on the full file as queue capacity grows. Class is used only to count frauds after ranking. The alternative keys are described in Section G.

E. Updating the queue over time

The notebook splits the file into 48 hourly windows and 288 ten-minute windows. After each window, it publishes the best K records seen so far. The current implementation compares (a) rebuilding top-K from all history, (b) updating a persistent heap, and (c) sorting the new window and merging it with the saved queue.

Window schedule	Rebuild from history	Persistent heap	Window sort + merge
48 windows	1.8638 s	0.0840 s	1.6977 s
288 windows	11.3013 s	0.1333 s	1.3624 s

Table 4. Fresh single-run cumulative times; K = 53. Strategy (a) currently rebuilds with a top-K heap, rather than full-sorting history. That distinction matters when comparing these results with the original problem statement.

A persistent heap keeps O(K) records between windows. Rebuilding retains O(n) history, while window sorting needs space for the incoming window and its merge buffer. Publishing an ordered heap queue also creates O(K) temporary output; K records is not a complete byte-level memory measurement.

F. Mathematical analysis

M1. Insertion sort and inversions

Let I be the number of pairs that are out of order. Each shift removes one inversion, and the outer loop visits n items, so $T(n) = \Theta(n + I)$. The best case is $\Theta(n)$, and the average and worst cases are $\Theta(n^2)$. The earlier C5 analysis reports I = 262,763,679 for seed 2514; this explains why 1% displacement can still be expensive.

M2. Merge sort

For $T(n) = 2T(n/2) + cn$, repeated substitution gives $T(n) = 2^jT(n/2^j) + jcn$. At $j = \log_2 n$, this becomes $nT(1) + cn \log_2 n = \Theta(n \log n)$. Master's theorem gives the same answer: $a = b = 2$ and $f(n) = \Theta(n)$, so Case 2 applies.

M3. Quicksort

With an extreme pivot, $T(n) = T(n - 1) + cn$. Unrolling adds $c[n + (n - 1) + ... + 2]$, giving $\Theta(n^2)$. With balanced partitions, the recursion tree has $\log_2 n$ levels and $\Theta(n)$ work per level, giving $\Theta(n \log n)$.

For a fixed 1:9 split, $T(n) = T(n/10) + T(9n/10) + cn$. The shortest and longest branches both have logarithmic depth. Each level costs at most cn, giving $O(n \log n)$; before the shortest branch ends, there are $\Theta(\log n)$ levels carrying $\Theta(n)$ work, giving the matching lower bound. The result is $\Theta(n \log n)$.

M4. Building and sorting a heap

Most heap nodes are close to the leaves and require little sift-down work. Summing by height gives $O(n \times \Sigma\, h/2^h) = O(n)$, because the series converges to 2. Heap construction is linear. The remaining n - 1 extractions each take $O(\log n)$, so total sorting time is $O(n \log n)$.

M5. Top-K cost and crossover

Full merge sort costs $O(n \log n)$ plus $O(K)$ to copy the queue. Heap selection costs $O(n \log K)$, followed by $O(K \log K)$ to publish it in order. The bounded buffer has a linear rejection scan, but a qualifying insertion can shift K items, giving $O(nK)$ worst-case time. The crossover depends on constants and the number of accepted replacements; a universal threshold such as $K = 0.35n$ does not follow from these bounds.

M6. Equal-band comparisons in naive quicksort

In a band containing m records, each pair is compared once by strict Lomuto partitioning: $m(m - 1)/2$ comparisons. Summing the actual group sizes gives 12,927,671,903 within-band comparisons. Comparisons between different bands add to this total and depend on pivot order. Therefore, the group sizes alone do not determine the exact total comparison count. Saved large C6 totals are extrapolations, not direct full-file measurements.

M7. Total work across w windows

Assume equal window size $m = n/w$. Re-sorting all accumulated history costs $\Sigma\, O(im \log(im)) = O(nw \log n)$. Rebuilding top-K from history, as the current notebook does, instead costs $O(nw \log K + wK \log K)$. A persistent heap costs $O(n \log K + wK \log K)$, including ordered publication. Window sort and merge costs $O(n \log(n/w) + wK)$.

M8. Extra space

Insertion and heap sort use $O(1)$ extra space. Merge sort uses $O(n)$. Naive quicksort can use $O(n)$ stack space; the hardened smaller-side recursion uses $O(\log n)$. Both top-K selectors use $O(K)$, including their output. These bounds describe growth, not the size of Python objects in bytes.

G. Changes to the ranking rule

I compared two local change scenarios. Freshness reverses only the Time tie-breaker. Continuous risk inserts risk descending between band and Amount. Both retain txn_id as the final tie-breaker. They change queue membership and recall, while leaving the algorithms' asymptotic bounds unchanged. These scenarios still need to be matched to the actual course-portal change request.

Rule	Frauds at K = 53	Smallest K for 50% recall
Standard	0	2,068
Freshness	0	2,086
Continuous risk	4	1,928

H. My recommendations

Overnight batch: I would use merge sort when an O(n) buffer is available. Its saved results are consistent across the six conditions, and it preserves equal-key order. If memory is too limited for the buffer, heap sort is a reasonable fallback.

Shift queue: I would use a size-K heap when K is small relative to the file and predictable replacement cost matters. The buffer is also worth measuring for very small K, because fewer comparisons can offset its shifts. For a large queue, compare both with full sorting rather than assuming a fixed crossover.

Live refresh: I would keep a persistent heap while scores remain fixed and the queue covers all records seen so far. If old scores change or old transactions expire, retaining only K survivors is insufficient. Historical records must be available for rescoring or replacement; simply merging the next window does not solve this.

Bonus. Analyst Console

The console is a separate, on-demand demonstration. It lets the analyst choose an algorithm, condition, sample size, K, and ranking rule, then shows the queue and run metrics. It does not replay the live feed. Run **python console.py** and open **http://127.0.0.1:8050**. The Python server and console.html use no external web framework.

Appendix. Earlier prediction log and reflections

Input	Earlier prediction	Reflection against saved results
C1	Merge wins; insertion loses.	Naive quicksort is actually fastest on the full file. Merge remains the safer batch choice.
C2	Merge wins; insertion loses.	Naive quicksort again has the lowest saved time; insertion grows quadratically.
C3	Insertion wins; naive quicksort loses.	Matches the saved results. An iterative stack prevents a crash, not quadratic work.
C4	Merge wins; insertion/naive quicksort lose.	Merge is fastest among completed full-file runs. Both quadratic cases are extrapolated DNF.
C5	Insertion wins; naive quicksort loses.	Merge is fastest at full size. Insertion needs about 115 seconds despite only 1% displacement.
C6	Hardened quicksort wins; naive quicksort loses.	Hardened quicksort finishes in 0.191 seconds; three-way partitioning skips equal-band groups.

Sources and reproducibility

Assignment brief: The Midnight Queue - Assignment 2. Data: ULB Credit Card Fraud Detection, creditcard.csv. Supporting work: results.csv, the three project notebooks, and the mathematical notes in reports/. General references: Stack Overflow for coding questions and course material for algorithm theory. Fresh checks use Python 3.12.14; saved tournament results came from an earlier run environment. Run tests with python -m unittest discover -s tests -v.